

Modul A: Inteligentni sistemi za podporo odločanju

Domača naloga 4

Dominik Bernik¹

¹ Univerza v Ljubljani, Fakulteta za Elektrotehniko
E-pošta: db0975@student.uni-lj.si

1 Uvod

Pri tej domači nalogi smo bili zadolženi za modeliranje sistema na štiri različne načine. Sistem smo morali modelirati z linearnim modelom, fuzzy-local modelom, fuzzy-global modelom in s pomočjo nevronske mreže.

2 Metodologija

2.1 Zbiranje izhodnih podatkov

Na razpolago smo imeli funkcijo `pendulum_process_obf`, ki prejme vektor vhodnih signalov dolžine N ter spremenljivko T_s , ki predstavlja čas vzorčenja.

Za identifikacijo sistema smo najprej identificirali statično karakteristiko sistema s postavljanjem vhodne vrednosti u na statično vrednost med 0 in 3. Nato smo počakali, da se sistem ustalili in nato naredili meritev.

V postopku identifikacije smo naredili tudi eksperiment stopničastega odziva. Ta nam je omogočil, da smo identificirali čas dinamičnega dela odziva, preden se ustali, in potrebno vzorčno hitrost za pravilno identifikacijo. Pri tem eksperimentu smo ugotovili, da je za identifikacijo primeren vzorčni čas $T_s = 0.05$ sekunde, čas dinamičnega dela signala pa je znašal med 1 in 1.3 sekunde. Zato smo za namen modeliranja uporabili različne stopničaste odzive med vrednostjo 0.9 in 1.7. Pridobljeni podatki so bili nato uporabljeni v postopku identifikacije sistema.

2.2 Linearni model

Pri modeliranju z linearnim modelom smo uporabili reda velikosti $n = 2$ in $m = 2$ za vhodne in izhodne podatke. Zaradi takšne konfiguracije smo dobili naslednji vektor vhodnih parametrov X :

$$X = [u(k-1), u(k-2), y(k-1), y(k-2), 1] \quad (1)$$

To je povzročilo, da so dimenzije vektorja parametrov θ enake $(5, 1)$. Za izračun parametrov θ je bila uporabljena metoda najmanjših kvadratov:

$$\theta = (X^T X)^{-1} X^T Y \quad (2)$$

V enačbi Y predstavlja vektor vseh izhodnih podatkov od indeksa n dalje.

Za napoved oziroma predikcijo $\hat{y}$ smo uporabili naslednjo enačbo:

$$\hat{y}(k) = X(k)\theta \quad (3)$$

Pridobljene podatke smo nato evalvirali s pomočjo metode RMSE (ang. root mean square error).

$$RMSE = \sqrt{\sum (\hat{y} - Y)^2} \quad (4)$$

Ter metode MSE (ang. mean square error).

$$MSE = \frac{1}{N} \sum (\hat{y} - Y)^2 \quad (5)$$

2.3 Mehki model (Fuzzy model Local/Global)

Za delovanje mehkega oziroma Fuzzy modela je ključnega pomena, da ugotovimo stopnjo pripadnosti posameznemu lokalnemu modelu. V ta namen uporabimo centre c_i , ki definirajo položaj posamezne funkcije pripadnosti v prostoru vhodnih spremenljivk.

Za izračun pripadnosti se lahko osredotočimo na dva možna pristopa. Prvi in najenostavnejši je izračun inverzne razdalje $\mu_i(x(k))$, ki predstavlja obratno sorazmerje razdalje trenutnega vhoda do posameznega centra i :

$$\mu_i^{\text{inv}}(x(k)) = \frac{1}{|x(k) - c_i|^m + \text{EPS}} \quad (6)$$

Zgornja enačba vsebuje majhno pozitivno konstanto $\text{EPS} = 10^{-6}$, s katero preprečimo deljenje z nič oziroma pojav $\frac{1}{0}$, kar bi povzročilo numerične težave. Enačba vsebuje tudi parameter mehkobe, fuzziness, $m > 0$.

Poleg inverzne razdalje poznamo tudi Gaussovo funkcijo pripadnosti, ki ni bila implementirana v tem delu:

$$\mu_i^{\text{Gauss}}(x(k)) = \exp\left(-\frac{(x(k) - c_i)^2}{2\sigma^2}\right) \quad (7)$$

Za pravilno delovanje modela je potrebno izvesti normalizacijo vseh vrednosti pripadnosti $\mu_i(x(k))$. To storimo s pomočjo naslednje enačbe, s čimer dobimo normalizirane uteži $\phi_i(x(k))$:

$$\phi_i(x(k)) = \frac{\mu_i(x(k))}{\sum_{j=1}^R \mu_j(x(k))} \quad (8)$$

V zgornji enačbi R predstavlja skupno število vseh centrov. Za preverjanje pravilnosti izračuna mora veljati naslednji pogoj:

$$\sum_{j=1}^R \phi_j(x(k)) = 1 \quad (9)$$

Končni izhod mehkega modela $\hat{y}(k)$ dobimo kot vsoto izhodov vseh R lokalnih linearnih modelov, pomnoženih z njihovimi pripadajočimi normaliziranimi utežmi:

$$y_i(k) = \theta_{i,0} + \theta_{i,1}x(k) \quad (10)$$

$$\hat{y}(k) = \sum_{i=1}^R \phi_i(x(k))y_i(k) \quad (11)$$

Pri identifikaciji parametrov θ pa se osredotočamo na dva različna pristopa, lokalni in globalni.

2.3.1 Lokalna identifikacija (Local)

Pri lokalnem pristopu parametre vsakega lokalnega modela ocenjujemo neodvisno od ostalih. Za vsako pravilo i definiramo regresijsko matriko $\bar{X}$, katere vrstice predstavljajo regresor $[1, x(k)]$, ter pripadajočo diagonalno matriko uteži W_i .

$$W_i = \text{diag}(\phi_i(x(1)), \phi_i(x(2)), \dots, \phi_i(x(N))) \quad (12)$$

Parametre posameznega lokalnega modela $\hat{\theta}_i^{\text{LOC}}$ nato izračunamo z uteženo metodo najmanjših kvadratov:

$$\hat{\theta}_i^{\text{LOC}} = (\bar{X}^T W_i \bar{X})^{-1} \bar{X}^T W_i Y \quad (13)$$

Ta postopek je lokalni, saj optimizacijo izvedemo ločeno za vsako izmed R pravil, pri čemer se upoštevajo le podatki v okolici centra c_i .

2.3.2 Globalna identifikacija (Global)

Pri globalnem pristopu parametre vseh lokalnih modelov ocenjujemo hkrati v enem samem koraku. Namesto ločenih matrik $\bar{X}$ sestavimo enotno, globalno regresijsko matriko Z . Ta matrika vključuje interakcije med vhodnimi podatki in normaliziranimi pripadnostmi vseh pravil:

$$Z = [\phi_1(x) \cdot \bar{X}, \phi_2(x) \cdot \bar{X}, \dots, \phi_R(x) \cdot \bar{X}] \quad (14)$$

Natančneje, posamezna vrstica matrike Z za vzorec k ima naslednjo obliko:

$$Z(k) = [\phi_1(k), \phi_1(k)x(k), \dots, \phi_R(k), \phi_R(k)x(k)] \quad (15)$$

Vektor vseh globalnih parametrov $\bar{\theta}^{\text{glb}}$, ki ima dimenzijo $(2R \times 1)$, nato izračunamo neposredno z običajno metodo najmanjših kvadratov:

$$\bar{\theta}^{\text{glb}} = (Z^T Z)^{-1} Z^T Y \quad (16)$$

Prednost globalnega pristopa je v tem, da minimizira celotno, globalno napako modela $\hat{y} - Y$ in bolje optimizira prehode med območji, vendar lahko pri tem izgubimo neposredno fizikalno interpretiranost posameznih lokalnih linearnih modelov.

2.4 Nevronske mreže

Zadnji način modeliranja sistema pa je s pomočjo nevronske mreže. Tu si lahko nevronske mreže predstavljamo kot nadgradnjo za linearne regresije.

Pri linearni regresiji iščemo direktno povezavo med vhodi in izhodi, nevronska mreža pa nam s pomočjo skritih plasti in nelinearnih funkcij omogoča, da modeliramo veliko bolj kompleksne sisteme.

Osnovni gradnik tu je nevron. Vsak nevron najprej naredi izračun, ki je v bistvu linearna regresija:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b \quad (17)$$

Kjer w predstavljajo uteži, b pa predstavlja bias. Glavna razlika od navadne linearne regresije pa je v tem, da ta rezultat pošljemo skozi aktivacijsko funkcijo, kot je npr. ReLU ali Tanh.

$$a = \sigma(z) \quad (18)$$

Za učenje pa smo se osredotočili na backpropagation, s pomočjo katerega smo posodobili uteži in bias v nevronske mreže. Za evalvacijo končnega modela smo tu uporabili MSE, ki smo ga že definirali zgoraj.

2.5 Gustafson-Kesselovo gručenje (GK)

Za avtomatsko določanje centrov c_i iz podatkov pogosto uporabimo algoritme gručenja. Klasični algoritem mehkih C-srednjih vrednosti uporablja standardno Evklidsko razdaljo, kar pomeni, da je prilagojen iskanju sferičnih (izotropnih) gručenj. Kadar pa podatki ležijo vzdolž nelinearne krivulje (npr. sinusne funkcije), sferične gruče niso optimalne, saj naravne gruče v tem primeru izgledajo kot kratki, usmerjeni tangentni odseki.

Gustafson in Kessel sta to pomanjkljivost odpravila tako, da sta vsaki gruči dodelila lastno mehko kovariančno matriko F_i , s katero definiramo prilagodljivo Mahalanobisovo razdaljo:

$$A_i = (\rho_i \det F_i)^{1/D} F_i^{-1} \quad (19)$$

$$d_{i,k}^2 = (z(k) - v_i)^T A_i (z(k) - v_i) \quad (20)$$

V zgornjih enačbah $z(k) = [x(k), y(k)]^T$ predstavlja skupni vhodno-izhodni vzorec podatkov v dvorazsežnem realnem prostoru, v_i je center i -te gruče, D pomeni dimenzijo podatkovnega prostora (v našem primeru $D = 2$), matrika A_i pa določa normo razdalje. Parameter ρ_i predstavlja volumen gruče, ki ga običajno nastavimo na $\rho_i = 1$, s čimer zagotovimo, da ostanejo volumni vseh gručenj primerljivi.

Cilj GK algoritma je minimizacija namenske kriterijske funkcije J_q , ki ima enako obliko kot pri FCM algoritmu, vendar uporablja podatkovno odvisno razdaljo:

$$J_q(U, V, \{F_i\}) = \sum_{i=1}^R \sum_{k=1}^N u_{i,k}^q d_{i,k}^2 \quad (21)$$

kjer je R izbrano število pravil (gruč), N je skupno število vzorcev, $q > 1$ je parameter mehkebe gručenja

(fuzzifier), $u_{i,k} \in [0, 1]$ pa predstavlja stopnjo pripadnosti k -tega vzorca i -ti gruči. Pri tem mora biti zadoščeno pogoju $\sum_{i=1}^R u_{i,k} = 1$ za vsak vzorec.

Minimizacijo izvedemo z iterativnim postopkom izmenične optimizacije, kjer v vsakem koraku posodablamo centre v_i , kovariančne matrike F_i in matrike pripadnosti $U = [u_{i,k}]$ po naslednjih enačbah:

$$v_i = \frac{\sum_{k=1}^N u_{i,k}^q z(k)}{\sum_{k=1}^N u_{i,k}^q} \quad (22)$$

$$F_i = \frac{\sum_{k=1}^N u_{i,k}^q (z(k) - v_i)(z(k) - v_i)^T}{\sum_{k=1}^N u_{i,k}^q} \quad (23)$$

$$u_{i,k} = \left(\sum_{j=1}^R \left(\frac{d_{i,k}^2}{d_{j,k}^2} \right)^{\frac{1}{q-1}} \right)^{-1} \quad (24)$$

Postopek ponavljamo, dokler se matrika pripadnosti U in centri V opazno ne spremenijo oziroma dokler algoritem ne konvergira.

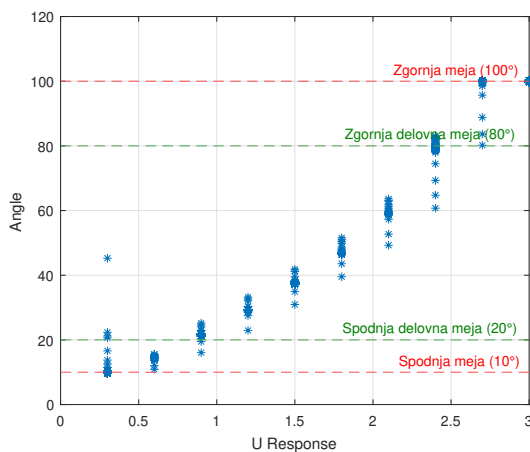
Ko dosežemo konvergenco, dobljene dvodimenzionalne prototipe v_i projiciramo na vhodno os x , s čimer dobimo iskane centre c_i za naš mehki model:

$$c_i^{gk} = (v_i)_x, \quad i = 1, \dots, R \quad (25)$$

Prednost tega postopka je, da algoritem samodejno določi optimalno postavitev centrov pravil neposredno iz podatkov, brez vnaprejšnjega poznavanja natančne matematične oblike nelinearnega procesa.

3 Rezultati

Pri modeliranju je bil prvi postopek, da naredimo analizo statične karakteristike:



Slika 1: Statična karakteristika sistema

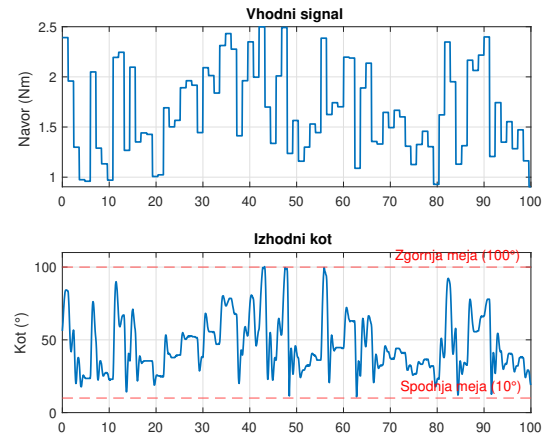
Iz slike 1 je razvidno, da za dobro modeliranje območja od 20 do 80 stopinj potrebujemo vhod držati med vrednostmi 0.7 in 2.3.

3.1 Linearni model

Za modeliranje linearnega modela smo najprej predpostavili red sistema. Tu smo določili $n = 2$ in $m = 2$, kar pomeni, da bomo imeli theta z dimenzijo (4,1). Prav tako moramo pripraviti pravilni vektor X dimenzije (4,(N-n)), kjer N predstavlja število vzorcev.

$$X(:, k) = [y(k-1), y(k-2), u(k), u(k-1)] \quad (26)$$

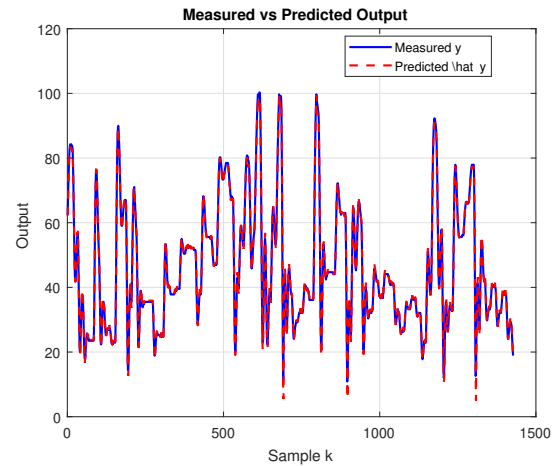
Za pridobitev podatkov smo naredili stopnjičasti vhodni signal med vrednostjo 0.7 in 2.3.



Slika 2: Vhodni in izhodni učni signal

Iz zgornjega grafa 2 smo vhodni signal shranili v parameter u in izhodni signal v y . Za izračun θ smo nato uporabili LS enačbo 2.

Po izračunu nove θ smo nato naredili validacijo delovanja, kjer smo prišli do naslednjega rezultata.



Slika 3: Prikaz referenčne vrednosti y in predikcije modela

Tu smo prav tako izračunali RMSE, kjer smo dobili izračunano napako $RMSE = 0.9205$ stopinj.

3.2 Mehki model (Fuzzy Model Local)

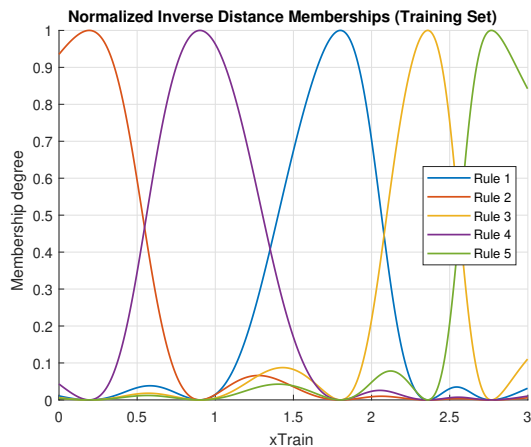
Pri modeliranju mehkega modela (ang. Fuzzy Model Local) smo najprej določili število zelenih centrov. Za na-

men reševanja te naloge smo jih izbrali 5. Za izračun lokacij tek centrov, smo uporabili metodo Gustafson-Kessel, kjer smo dobili naslednje rezultate.



Slika 4: Pripadnost modelu glede na izračunano pripadnost

To nam je omogočilo, da smo naredili uteženi LS za vsak posamezni center. Tu smo prav tako izračunali pripadnost funkciji glede na normalizirano inverzno razdaljo.

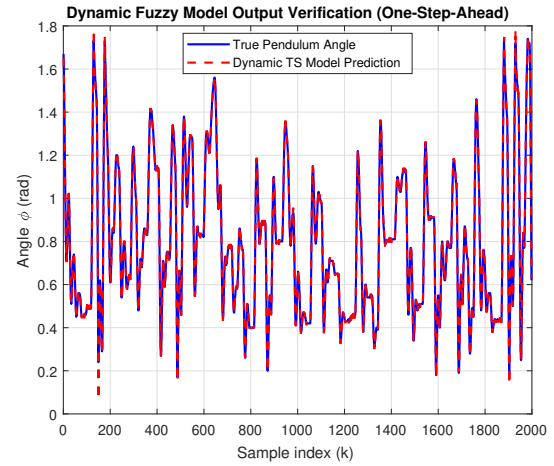


Slika 5: Normalizirana pripadnost posameznemu centru

Po izračunani pripadnosti smo nato izračunali θ za vsak posamezni center. Za izračun smo uporabili enako metodo vzorčenja kot pri izdelavi linearnega modela ter sestavo vektorja X .

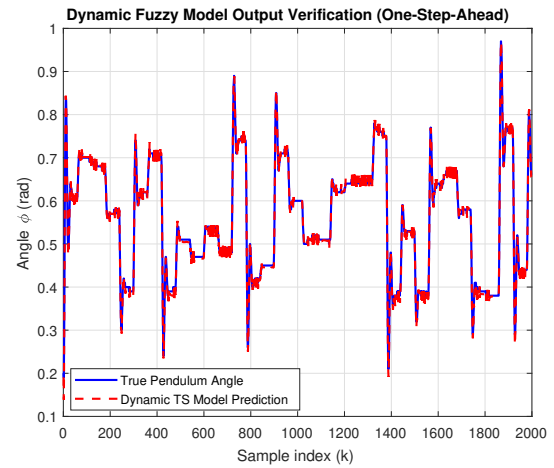
$$Xe = [u(k), u(k-1), y(k-1), y(k-2), 1] \quad (27)$$

Po postopku učenja smo na učni množici dobili $MSE = 0.352527$ stopinj, kar predstavlja boljši odziv kot pri linearnem modelu.



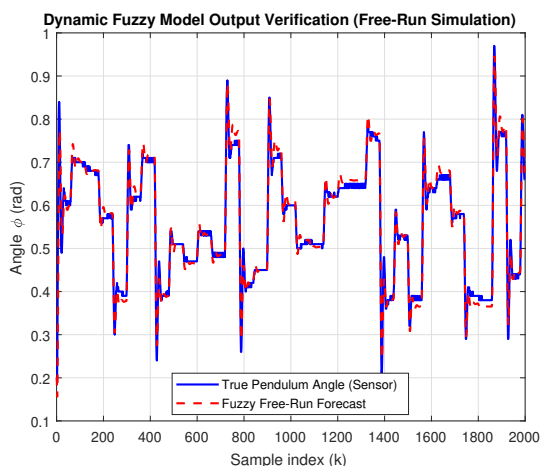
Slika 6: Prikaz referenčne vrednosti y in predikcije modela

Po učenju smo prav tako naredili validacijo, kjer smo pri one-step predikciji dosegli $MSE = 0.252158$ stopinj.



Slika 7: Validacija modela na testni množici, One step predikcija

Kot dodatni preizkus smo tu naredili tudi Free-run simulacijo,

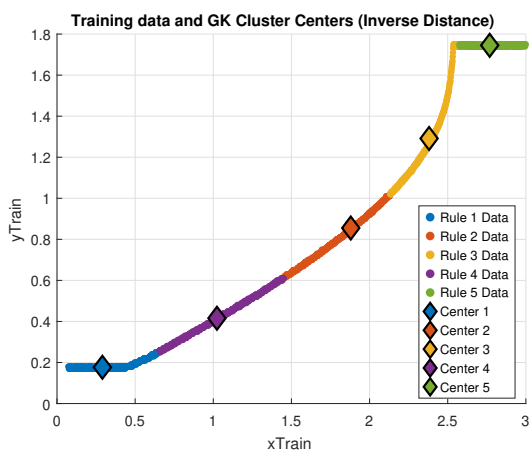


Slika 8: Validacija modela na testni množici, Free run simulacija

kjer smo dobili $MSE = 1.399721$ stopinj. Tu se moramo zavedati, da je Free-run simulacija odvisna samo od izračunanih $y(k-1)$ in $y(k-2)$, kar pomeni, da se napaka iz prejšnjih korakov prenaša naprej.

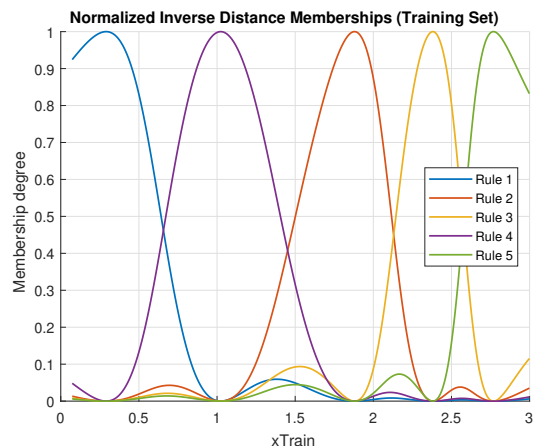
3.3 Mehki model (Fuzzy Model Global)

Pri izdelavi Fuzzy Global modela smo se odločili, da ga bomo naredili s pomočjo 5 centrov. Ponovno smo se obrnili na Gustafson-Kesselovo gručenje, kjer smo dobili naslednje rezultate:



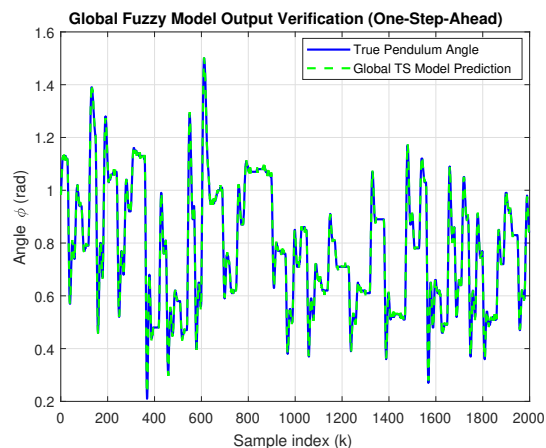
Slika 9: Postavitev centrov in pripadnost posameznemu pravilu

Po izračunu normalizirane inverzne uteži smo prišli do naslednjih ugotovitev pripadnosti:



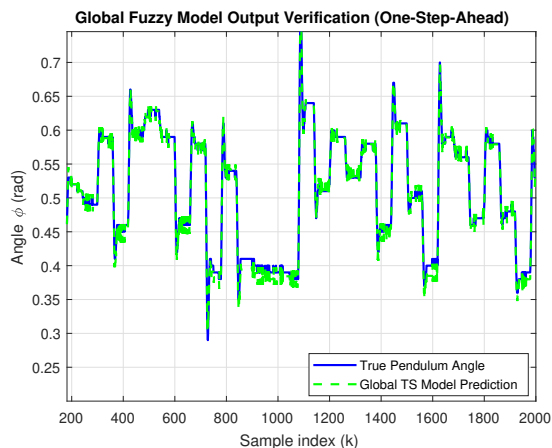
Slika 10: Pripadnost modelu glede na izračunano pripadnost

Naš naslednji postopek je bil ustvariti globalno matriko Z , kjer smo nato izračunali θ za vse modele hkrati. Po preizkusu na učni množici smo prišli do naslednjega rezultata:



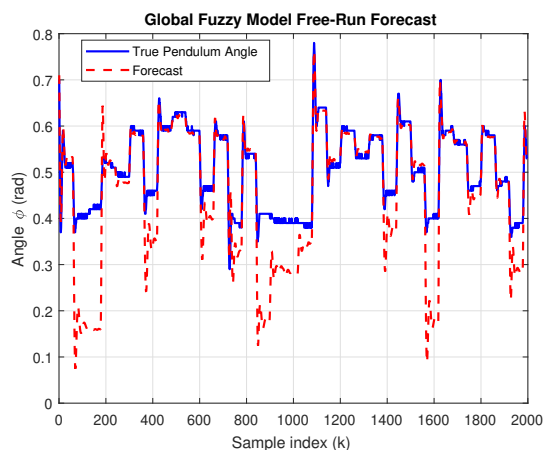
Slika 11: Predikcija modela na učni množici

Po izračunu smo prišli do $MSE = 0.147436$ stopinj, kar predstavlja boljši rezultat kot pri Fuzzy Local modelu, vsaj na učni množici. Za preverjanje delovanja smo se nato osredotočili na validacijo z One-Step predikcijo in Free-run simulacijo.



Slika 12: Validacija modela na testni množici, One step predikcija

Pri One-Step predikciji 12 smo prišli do $MSE = 0.305490$, kar predstavlja slabši rezultat kot pri Fuzzy Local modelu.



Slika 13: Validacija modela na testni množici, Free run simulacija

Pri Free-run simulaciji 13 pa smo dobili $MSE = 28.21$ stopinj, kar predstavlja slab rezultat. To se je zgodilo, ker v določenem delu nismo dovolj dobro zajeli karakteristike delovanja. To povzroči, da nam v določenem delu model razpade in da popolnoma napačne rezultate.

3.4 Nevronska mreža

Pri implementaciji nevronske mreže sem se najprej moral odločiti, kakšen bo vhodni vektor. Ker tu potrebujemo zajeti predhodno informacijo o stanju na vходу in predhodno stanje na izhodu, smo se odločili, da bo vhodni vektor naslednje oblike:

$$X(:, k) = [y(k-1), y(k-2), u(k), u(k-1)] \quad (28)$$

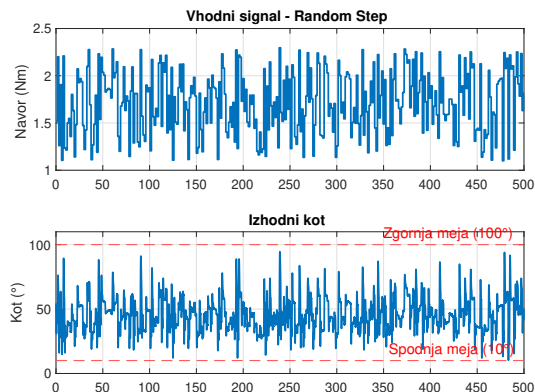
kjer $y(k)$ predstavlja izhod sistema, $u(k)$ pa vhodni signal. Na ta način omogočimo nevronske mreži, da se nauči dinamičnega obnašanja sistema.

Za učenje nevronske mreže sem uporabil arhitekturo z dvema skritima slojema po 16 nevroni v vsakem sloju.

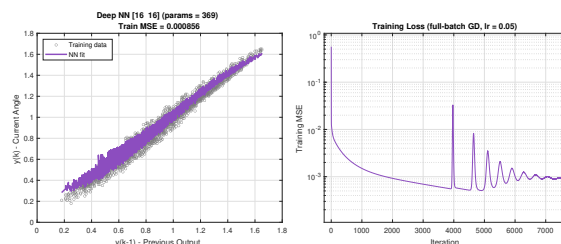
Za aktivacijsko funkcijo v skritih slojih smo izbrali hiperbolični tangens ($\tanh$).

Učenje je potekalo s polno paketno gradientno metodo (ang. full-batch gradient descent) z učno stopnjo $\eta = 0.05$ in 8000 ponovitvami. Kot izgubno funkcijo sem tu prav tako uporabil srednjo kvadratno napako. Začetne uteži pa smo inicializirali z naključnimi vrednostmi.

Pri učenju je bil uporabljen naslednji vhodni in izhodni signal:



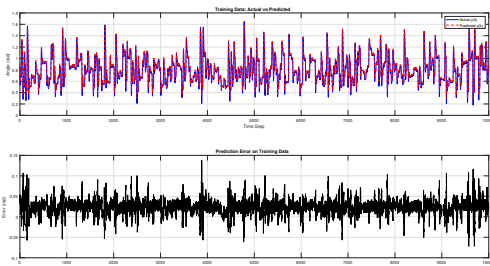
Slika 14: Trening podatki za nevronske mreže



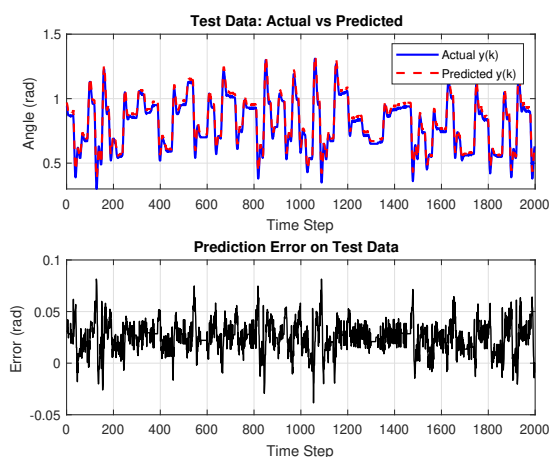
Slika 15: Prikaz učenja nevronske mreže, kjer je prikazana izguba na učni množici

Po izvajanju učenja, smo se osredotočili na training loss, ki je prikazan na grafu 15. Tu vidimo da z iteracijami izguba pada, kar pomeni, da se mreža uči in izboljšuje svoje napovedi. V grafu je tudi razvidno, da smo tekom učenja imeli nekaj špic, kar je posledica ne dobro definirane delovnega območja. To lahko popravimo s pomočjo boljše priprave učne množice ter z manjšanjem učne stopnje.

Z naučenim modelom smo nato naredili dva preizkusa, in sicer One-step predikcija na učni množici, kot baseline za primerjavo in nato One-step predikcija na testni množici, kjer smo dobili naslednje rezultate:



Slika 16: Prikaz One-step predikcije na učni množici



Slika 17: Prikaz One-step predikcije na testni množici

Iz zgornjih grafov 16 in 17 je razvidno, da je nevronska mreža zelo dober način modeliranja sistema, saj je MSE na učni množici znašal 0.000895 stopinj, na testni množici pa 2.967542 stopinj.

Tu bi pričakovali boljše rezultate, vendar je posledica tega rezultata naša izbira vhodnih podatkov, saj bi lahko uporabili več predhodnjih vhodnih in izhodnih podatkov, kar bi omogočilo nevronske mreži, da se bolje nauči kompleksna obnašanja sistema.

4 Zaključek

Na podlagi predstavljenih rezultatov lahko zaključimo, da se je kot najboljši model izkazala nevronska mreža, saj je dosegla daleč najnižjo napako na učni množici, kjer je MSE znašal 0.000895 stopinj. Kljub nekoliko višji napaki na testni množici, ki je znašala 2.967542 stopinj, bi se ta napaka lahko izboljšala s primerno pripravo vhodnih podatkov, na primer z uporabo večjega števila preteklih vzorcev na vходу in izhodu.

Na drugem mestu po uspešnosti se uvršča Fuzzy Local model, ki se je izkazal kot najbolj robusten, saj je pri one-step predikciji dosegel MSE = 0.252158 stopinj, pri

free-run simulaciji pa MSE = 1.399721 stopinj, kar pomeni, da se napaka sicer prenaša, vendar model ohrani stabilnost.

Fuzzy Global model sicer na učni množici dosega dobro napako (MSE = 0.147436 stopinj), vendar se pri free-run simulaciji izkaže za nestabilnega, saj napaka znaša kar 28.21 stopinj, kar pomeni, da model v določenih delih razpade in daje popolnoma napačne rezultate.

Linearni model je pričakovano najslabši z RMSE = 0.9205 stopinj, kar potrjuje, da linearni pristop ne zmore dovolj natančno opisati nelinearnega obnašanja sistema.